LLM & Agentic Systems

Assignment: Small-Model Sequential Instruction Tuning + Judge Pipeline

LLM & Agentic Systems — Graduate Course

Assignment 3: Sequential Instruction Tuning of a Small LLM with Strong-Model Judge Evaluation

Two-Stage Post-Training Alignment on UTSA HPC

Dr. Peyman Najafirad (Paul Rad)
TA: Mohammad Bahrami

Field	Details
Due Date	April 6th, 2026 — 23:59 (11:59 PM)
Format	Individual project — Code repository + GitHub blog post (5 pages + prompt appendix)
Submission	GitHub blog post (5 pages + prompt appendix) published on your repository, plus code and all artifacts, submitted via course portal

1. Overview & Motivation

In the previous assignment, you studied how multi-agent debate and judge-based reasoning can improve answer quality beyond a single direct response. In this assignment, we move from inference-time reasoning systems to post-training alignment systems. You will build a two-stage instruction-tuning pipeline for a small language model, run training on UTSA HPC, and evaluate the resulting model after each stage using both automatic metrics and a stronger LLM judge.

The central idea is that smaller models are often efficient and deployable, but they are limited by weak instruction following, unreliable formatting, and inconsistent behavior on specialized tasks. A common post-training approach is to fine-tune on general instruction data first, then add structured-output supervision in a second stage using imitation learning, where a stronger teacher model generates high-quality outputs and the smaller student model learns to replicate that behavior. A critical open question in this workflow is whether the second stage of imitation-based training preserves or degrades the general capabilities gained in the first stage.

Your task is to implement this two-stage pipeline end to end and measure what happens at each stage.

The core research question you will investigate:

If you first fine-tune a small LLM on Alpaca-style instruction data and then continue fine-tuning on a JSON-structured instruction dataset created through imitation learning from a stronger teacher model, does the model gain structured-output reliability while maintaining its general instruction-following ability, or does catastrophic forgetting degrade the gains from the first stage?

This assignment is intentionally designed to reflect real-world post-training practice: sequential fine-tuning stages, imitation learning from a stronger model for data construction, separate evaluation at each checkpoint, forgetting analysis, and LLM-as-a-Judge evaluation.

2. Problem Definition

You will implement and evaluate a two-stage instruction-tuning pipeline on UTSA HPC. The system consists of the following components:

- **Student Model:** A small instruction-tunable LLM that will be fine-tuned across two stages.
- **Stage 1 Training Data:** Alpaca-style general instruction-following examples.
- **Stage 2 Training Data (Teacher-Generated):** A JSON-structured instruction dataset that you create through imitation learning, using a stronger teacher model (e.g., Llama 3.1 70B Instruct) to generate high-quality structured responses that the student model will learn to replicate.
- **Fine-Tuning Pipeline:** A QLoRA-based training workflow executed sequentially on UTSA HPC: first on Alpaca data, then continued on the teacher-generated JSON Instruct data.
- **Judge Model:** A stronger LLM that evaluates and compares outputs at each checkpoint.

2.1 Student Model

Select one small model from the following class of models suitable for UTSA HPC fine-tuning:

- Phi-3.5 Mini

- Llama 3.2 3B
- Qwen2.5 3B
- Gemma 2 2B

You must justify your selection in the report.

Recommended default: Phi-3.5 Mini Instruct, due to its strong small-model performance and practical suitability for QLoRA-based post-training.

2.2 Stage 1 Data: Alpaca Instruction Data

The first training stage uses Alpaca-style general instruction-following examples. These are prompt/response pairs covering a broad range of tasks such as open-ended generation, rewriting, brainstorming, summarization, and simple QA. You may use the original Stanford Alpaca dataset or a comparable cleaned variant (e.g., Alpaca-Cleaned, Dolly, or a filtered subset of Open Assistant).

The purpose of Stage 1 is to establish a strong general instruction-following baseline before any structured-output specialization.

2.3 Stage 2 Data: Teacher-Generated JSON Instruct Dataset

The second training stage uses a structured JSON instruction dataset that **you create through imitation learning from a stronger model**. This means you design the task prompts, feed them to a stronger teacher model such as **Llama 3.1 70B Instruct**, collect the teacher's high-quality structured responses, and use those responses as gold-standard training supervision for the smaller student model. The student learns to replicate the teacher's structured-output behavior without ever interacting with the teacher at inference time.

Note that this is **not** classical knowledge distillation in the Hinton et al. (2015) sense, which requires access to the teacher's soft probability distributions (logits) and uses KL divergence as the training objective. Here, the student only sees the teacher's final text outputs and trains on them using standard supervised fine-tuning (cross-entropy loss). This approach is more precisely called imitation learning, synthetic data generation, or black-box distillation.

Imitation learning from a stronger model is important because the student model on its own may produce poorly formatted, incomplete, or invalid JSON. By training on outputs generated by a much stronger model, the student can learn formatting discipline, schema adherence, and output consistency that it could not acquire from its own generations.

Your imitation learning pipeline must follow these steps:

- Design a diverse set of task prompts covering the five required structured-output task types (listed below).
- Feed each prompt to the teacher model (**Llama 3.1 70B Instruct** or comparable) and collect the generated response.
- Validate every teacher response for JSON correctness. If a response contains invalid JSON, discard it and regenerate.
- Pair each validated teacher response with the original prompt to form a training example in the standard (instruction, input, output) schema.
- The result is your teacher-generated training set: prompts authored by you, responses generated by the stronger model.

The teacher-generated JSON Instruct dataset should cover at least the following task types:

- JSON extraction from unstructured text (e.g., extract entities, dates, or attributes into a JSON object)
- Schema-constrained generation (e.g., given a schema, produce a valid JSON object that conforms to it)
- Exact-label classification with JSON output (e.g., classify a text and return the result as a JSON object with a fixed set of allowed labels)
- JSON repair or formatting correction (e.g., fix malformed JSON into valid JSON)
- Tool-call argument generation (e.g., produce a JSON object representing a function call with named parameters)

Each training example must follow a consistent schema with an instruction field, an optional input field, and an output field containing valid JSON. All outputs must be validated for JSON correctness before inclusion in the training set.

2.4 Evaluation Scope

You must evaluate the model at three checkpoints:

- **Checkpoint 0:** The original untuned student model (baseline).
- **Checkpoint 1:** The model after Stage 1 (Alpaca-only fine-tuning).
- **Checkpoint 2:** The model after Stage 2 (fine-tuning on teacher-generated JSON Instruct data on top of the Alpaca checkpoint).

At each checkpoint, you run two evaluation suites:

- **Alpaca Evaluation:** A held-out set of general instruction-following prompts evaluated using the methodology from the original Alpaca paper (Taori et al., 2023). This uses pairwise comparison by a stronger judge model, following the Self-Instruct evaluation protocol.
- **JSON Evaluation:** A held-out set of structured-output prompts evaluated for JSON validity, schema compliance, exact-match accuracy, and field-level correctness.

Running both evaluation suites at all three checkpoints is what allows you to answer the core research question: does Stage 2 training preserve or degrade the Alpaca capabilities gained in Stage 1?

3. Required System Architecture

Your pipeline must implement the following sequential post-training workflow.

Phase 1 — Data Construction

1a. Alpaca Data Preparation

- Select or download an Alpaca-style instruction dataset.
- Clean and normalize all examples into a unified training schema (instruction, input, output).
- Validate format consistency and remove malformed or low-quality samples.
- Set aside a held-out evaluation subset that will never be used for training.

1b. Teacher-Generated JSON Instruct Data Construction

- Design a diverse set of structured-output task prompts covering the five required task types (JSON extraction, schema-constrained generation, exact-label classification, JSON repair, tool-call argument generation).
- **Run imitation learning generation:** Feed each prompt to the teacher model (**Llama 3.1 70B Instruct** or comparable) and collect the generated responses. These teacher outputs become the training targets for the student.
- Validate every teacher response for JSON correctness. Discard or regenerate any response that contains invalid JSON.
- Pair each validated teacher response with the original prompt to form a complete training example.
- Normalize all teacher-generated examples into the same training schema used for Alpaca data (instruction, input, output).
- Set aside a held-out JSON evaluation subset that will never be used for training.

Phase 2 — Stage 1: Alpaca Fine-Tuning on UTSA HPC

- Fine-tune the student model on the Alpaca instruction dataset using QLoRA.
- Run the training job on UTSA HPC using GPU resources.
- Log training configuration, GPU allocation, checkpoints, loss curves, and final adapter outputs.
- **Save the Stage 1 adapter checkpoint.** This checkpoint becomes the starting point for Stage 2.
- Run Checkpoint 1 evaluation: evaluate this model on both the Alpaca held-out set and the JSON held-out set. Record all scores.

Phase 3 — Stage 2: Fine-Tuning on Teacher-Generated JSON Data on UTSA HPC

- **Load the Stage 1 checkpoint** (Alpaca-tuned model) as the starting point.
- Continue fine-tuning on the teacher-generated JSON Instruct dataset using QLoRA.
- Run the training job on UTSA HPC. Log all training details as in Stage 1.
- **Save the Stage 2 adapter checkpoint.**

- Run Checkpoint 2 evaluation: evaluate this model on both the Alpaca held-out set and the JSON held-out set. Record all scores.

Phase 4 — Judge Evaluation and Forgetting Analysis

- At each of the three checkpoints (untuned, after Stage 1, after Stage 2), generate responses to the full Alpaca held-out set and the full JSON held-out set.
- For the Alpaca evaluation, use the methodology from the Alpaca paper: present the judge model with pairwise responses and ask it to select the better one. Follow the Self-Instruct evaluation protocol with at least 100 evaluation prompts or a comparable held-out instruction set.
- For the JSON evaluation, compute automatic metrics (JSON validity, schema compliance, exact match) and additionally use the judge for qualitative scoring.
- The judge must score both responses across multiple dimensions:
  - Instruction Following
  - Correctness
  - Clarity
  - Completeness
  - Structured Output Validity (for JSON tasks)
  - Hallucination / Fabrication Risk
- To reduce ordering bias, randomize or swap response order when presenting outputs to the judge.
- **Forgetting analysis:** Compare the Alpaca evaluation scores at Checkpoint 1 vs Checkpoint 2. If scores drop, quantify the regression. If scores hold or improve, document that finding. This comparison is the central result of the assignment.

4. Required Experiments

You must conduct the following experiments and report quantitative results. Use any accessible open or API-based models, but document all model names, versions, and hyperparameters.

4.1 Three-Checkpoint Comparison

This is the primary experiment. You must compare the model at all three checkpoints on both evaluation suites.

Model Checkpoint	Alpaca Judge Win Rate	ROUGE-L / BERTScore	JSON Validity	Schema Compliance	Exact Match
Checkpoint 0: Untuned base	(expected low)	(expected low)	(expected low)	(expected low)	(expected low)
Checkpoint 1: After Stage 1 (Alpaca)	(expected high)	(expected high)	(low/medium)	(expected low)	(expected low)
Checkpoint 2: After Stage 2 (Teacher JSON)	(maintained? dropped?)	(maintained? dropped?)	(expected high)	(expected high)	(expected high)

You must fill in this table with actual measured values. The key finding is what happens to the Alpaca columns at Checkpoint 2.

4.2 Alpaca Evaluation (Self-Instruct Protocol)

Following the methodology from Taori et al. (2023), evaluate general instruction-following quality using pairwise comparison:

- Use a held-out set of at least 100 general instruction prompts (drawn from the Self-Instruct evaluation set or a comparable source).
- At each checkpoint, generate responses to all held-out prompts.
- Present pairwise responses to the judge model (e.g., Checkpoint 0 vs Checkpoint 1, then Checkpoint 1 vs Checkpoint 2).
- The judge selects the better response or declares a tie.
- Report win rate, tie rate, and average judge scores per dimension.

In addition to judge-based evaluation, compute the following automatic metrics against reference answers where available:

- ROUGE-1, ROUGE-2, ROUGE-L
- BERTScore
- Output length (average tokens per response)
- Task completion rate (did the model follow the instruction at all)

4.3 JSON Structured Output Evaluation

Construct a held-out JSON benchmark with at least 100 prompts covering the five required task types. At each checkpoint, generate responses and compute:

- **JSON validity rate:** percentage of outputs that parse as valid JSON.
- **Schema compliance rate:** percentage of valid JSON outputs that match the required schema (correct keys, correct value types).
- **Exact-match accuracy:** percentage of outputs that exactly match the expected JSON output.
- **Field-level F1:** for extraction tasks, measure precision and recall at the individual field level.
- **Common error taxonomy:** categorize and count the most frequent formatting errors (missing brackets, wrong types, extra fields, truncated output).

4.4 Forgetting Analysis

This is the central analytical contribution of the assignment. You must directly compare the Alpaca evaluation results at Checkpoint 1 (after Stage 1 only) against Checkpoint 2 (after both stages).

Report:

- The absolute change in Alpaca judge win rate from Checkpoint 1 to Checkpoint 2.
- The absolute change in ROUGE-L and BERTScore from Checkpoint 1 to Checkpoint 2.
- Per-category breakdown: did forgetting affect some instruction types more than others (e.g., open-ended generation vs summarization vs short QA)?
- Representative examples where the model regressed after Stage 2 (if any).
- Representative examples where the model improved or held steady after Stage 2.

If you observe catastrophic forgetting, discuss possible causes (learning rate too high, too many epochs, insufficient data diversity). If you do not observe forgetting, explain what may have prevented it.

4.5 Ablation Study

You must include at least one ablation from the following:

- Vary the number of Stage 2 training epochs (e.g., 1 vs 2 vs 3) and measure how forgetting scales with additional training.
- Vary the Stage 2 learning rate (e.g., 2e-5 vs 1e-5 vs 5e-6) and measure its effect on both JSON accuracy and Alpaca retention.
- Reduce the Stage 2 dataset size (e.g., 50%, 25%) and measure the tradeoff between JSON improvement and forgetting.
- Compare sequential two-stage training against a single combined training run on the merged Alpaca + teacher-generated JSON data.

The purpose is to identify which training decisions most strongly influence the forgetting/retention tradeoff.

5. Written Report (GitHub Blog Post)

Instead of a traditional PDF report, publish your findings as a blog post on your GitHub repository. Use either a GitHub Pages site or a well-formatted README/Markdown file in your repo (for example, REPORT.md). The blog post should be 5 pages equivalent in length plus appendix and structured as follows.

Section	Content	Approx. Length
1. Methodology	Student model choice, Alpaca data source, imitation learning pipeline for JSON Instruct dataset, teacher model setup, training design for both stages, UTSA HPC setup, judge model choice, evaluation protocol, hyperparameters	1 page
2. Experiments	Three-checkpoint comparison table, Alpaca evaluation results, JSON evaluation results, forgetting analysis, ablation results, tables and figures	3 pages
3. Analysis	Qualitative comparison of outputs across checkpoints, failure case analysis, discussion of forgetting vs retention, what the results imply about sequential fine-tuning	1 page
4. Prompt Engineering	How you designed prompts for teacher generation and judge evaluation, and how prompts changed after failure analysis	included in report
Appendix: Full Prompts	Final prompt templates for teacher-model JSON generation (imitation learning), student formatting, and judge evaluation	No page limit

Your report must clearly connect experimental observations to the broader post-training and alignment concepts discussed in class, especially catastrophic forgetting, sequential fine-tuning, imitation learning from stronger models, and the role of data composition in post-training.

6. Code Repository Requirements

Your GitHub repository must include:

- README.md with setup instructions, dependencies, and reproduction steps
- Modular code with separate components for:
  - Alpaca data preparation,
  - JSON Instruct dataset construction (including teacher-model generation scripts),
  - Stage 1 training,
  - Stage 2 training,
  - inference at each checkpoint,
  - judge evaluation,
  - and result aggregation

- A configuration file containing:
  - model names,
  - learning rate for each stage,
  - LoRA parameters,
  - max tokens,
  - number of epochs per stage,
  - batch size,
  - and evaluation settings
- Prompt templates stored as editable templates, not hardcoded inline
- Logs and artifacts saved for every run, including:
  - Checkpoint 0 (baseline) outputs,
  - Checkpoint 1 (Alpaca-tuned) outputs,
  - Checkpoint 2 (teacher-generated JSON-tuned) outputs,
  - judge scores at each checkpoint,
  - training logs for both stages
- Evaluation scripts that reproduce all tables and figures in the blog post
- requirements.txt or environment.yml for reproducibility
- At least one UTSA HPC batch script used to launch training for each stage

This repository expectation closely follows the structure used in Assignment 2, including modular components, config separation, logging, and reproducibility.

7. Grading Rubric

Component	Weight	Criteria
Running System	30%	Working two-stage instruction-tuning pipeline; clean code; reproducible training and evaluation at all three checkpoints; proper artifact logging
UTSA HPC Training Pipeline	15%	Successful GPU training jobs for both Stage 1 and Stage 2 on UTSA HPC; correct scheduler usage; documented environment and resource configuration
Write-up Blog Post	40%	Clear writing; strong structure; rigorous three-checkpoint comparison; meaningful forgetting analysis; well-formatted Markdown with tables/figures; solid ablation results
Prompt Engineering	15%	Well-designed prompts for teacher-model JSON generation (imitation learning); thoughtful judge prompts; evidence of prompt iteration and improvement

8. Suggested Technical Configuration

The following is a recommended default configuration for students who want a concrete starting point:

Parameter	Value
Student model	Phi-3.5 Mini Instruct
Fine-tuning method	QLoRA
Precision	4-bit
Stage 1 epochs	2–3
Stage 2 epochs	2–3
Learning rate (both stages)	2e-5
Max sequence length	1024 or 2048
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Teacher model (for imitation learning)	Llama 3.1 70B Instruct
Judge model	Llama 3.1 70B Instruct (or comparably strong model)
Alpaca evaluation prompts	At least 100 (Self-Instruct eval set or comparable)
JSON evaluation prompts	At least 100 (covering all five task types)

These defaults are recommendations only. You may deviate from them if justified in your report.

9. Example Judge Output Format

Your judge should return a structured output such as:

```
{
  "prompt_id": "alpaca_eval_042",
  "checkpoint_a": "checkpoint_1_alpaca",
  "checkpoint_b": "checkpoint_2_teacher_json",
  "response_a_scores": {
    "instruction_following": 5,
    "correctness": 4,
    "clarity": 5,
    "completeness": 4,
    "structured_output_validity": 3,
    "hallucination_risk": 4
  },
  "response_b_scores": {
    "instruction_following": 4,
    "correctness": 4,
    "clarity": 4,
    "completeness": 3,
    "structured_output_validity": 5,
    "hallucination_risk": 5
  },
  "winner": "A",
  "justification": "Response A followed the open-ended
    instruction more naturally. Response B showed stronger
    formatting discipline but was slightly less complete
    on the general instruction task."
}
```

You may design your own schema, but it must be structured, reproducible, and easy to aggregate. Include the checkpoint identifiers so that results can be grouped by comparison pair (Checkpoint 0 vs 1, Checkpoint 1 vs 2, Checkpoint 0 vs 2).

10. Key References

The following references are relevant to this assignment and align with the topics covered in lecture:

[1] Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models.

[2] Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs.

[3] Taori, R. et al. (2023). Alpaca: A Strong, Replicable Instruction-Following Model.

[4] Wang, Y. et al. (2023). Self-Instruct: Aligning Language Models with Self-Generated Instructions.

[5] Gu, J. et al. (2024). A Survey on LLM-as-a-Judge.

[6] Kenton, Z. et al. (2024). On Scalable Oversight with Weak LLMs Judging Strong LLMs.

[7] Rafailov, R. et al. (2024). From Human Preferences to Post-Training Alignment Pipelines.